



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

N° de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 26 de Septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Descripción

En esta práctica se trabajó con la importación y manipulación de dos modelos tridimensionales: un Holocrón y un satélite. Ambos modelos fueron preparados previamente en Blender, separando las piezas que requerían movimiento para posteriormente importarlas de manera individual al proyecto.

El objetivo principal fue utilizar nuevamente el modelado jerárquico para mantener una relación entre los objetos principales y sus componentes. En el Holocrón se importó un cuerpo central junto con ocho esquinas independientes, mientras que el satélite se dividió en un cuerpo principal y cuatro alas.

También se agregaron controles mediante teclado para modificar individualmente las esquinas del Holocrón, desplazar el satélite sobre los ejes X, Y y Z y rotar sus diferentes alas desde sus puntos de unión.

Desarrollo

Preparación e importación de los modelos

Para poder aplicar movimientos independientes, los modelos fueron separados previamente en Blender y exportados como archivos FBX.

En el caso del Holocrón se utilizaron nueve archivos:

- cuerpoholocron.fbx
- triangulosup1.fbx
- triangulosup2.fbx
- triangulosup3.fbx
- triangulosup4.fbx
- trianguloinf1.fbx
- trianguloinf2.fbx
- trianguloinf3.fbx
- trianguloinf4.fbx

Para el satélite se utilizaron:

- cuerposatelite.fbx
- alader1.fbx
- alader2.fbx
- alaizq1.fbx

- alaizq2.fbx

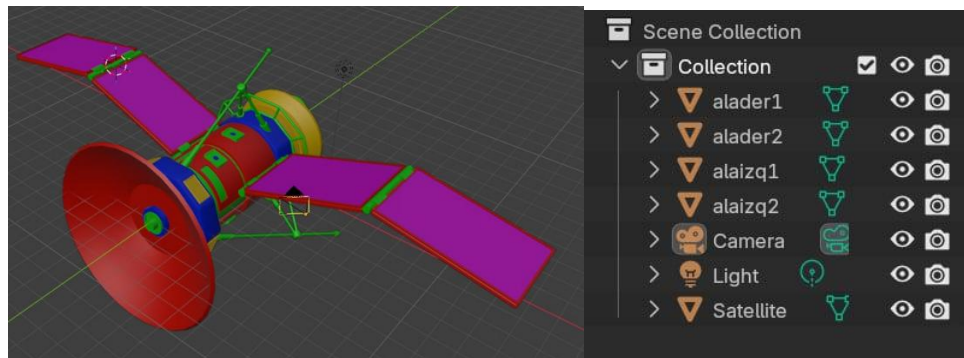


Figura 1. Separación de las partes del satélite en Blender.

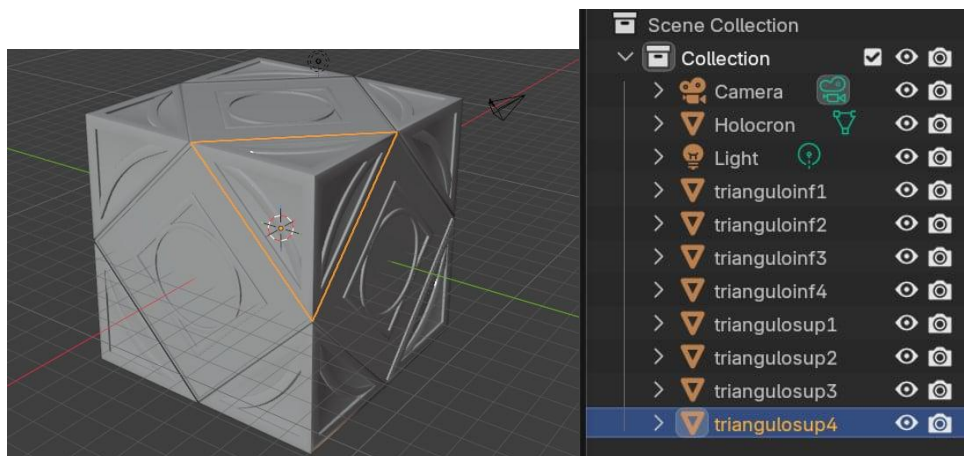


Figura 2. Separación de las partes del holocron en Blender.

Importación del Holocrón con jerarquía

Para el Holocrón se utilizó un cuerpo principal como objeto padre y ocho esquinas como elementos independientes.

El cuerpo completo parte de una matriz llamada holocronPadre, que establece su posición y escala dentro de la escena. Las ocho esquinas parten de esta misma matriz, por lo que mantienen una relación con el cuerpo central.

```

603 //holocron, padre comun situado al lado del Rover
604 //las esquinas son hijas del centro, no de una bisagra en su propio borde
605 glm::mat4 holocronPadre = glm::mat4(1.0f);
606 holocronPadre = glm::translate(holocronPadre, glm::vec3(-18.0f, 6.0f, -1.5f));
607 holocronPadre = glm::scale(holocronPadre, glm::vec3(0.8f));
608
609 color = glm::vec3(0.05f, 0.30f, 0.50f);
610 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
611 model = glm::rotate(holocronPadre, glm::radians(-180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
612 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
613 CuerpoHolocron_M.RenderModel();
614
615 //origen comun de los ocho fbx respecto al cuerpo
616 glm::vec3 origenEsquinas = glm::vec3(0.00407529f, 0.04012322f, 0.01084948f);
617 color = glm::vec3(0.85f, 0.60f, 0.15f);
618 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
619
620 //el giro propio conserva la trayectoria del centro y completa una vuelta por ciclo
621 GLfloat aperturaHolocron = 0.0f;
622

```

Figura 3. Holocrón importado y reconstruido a partir de sus diferentes piezas.

Rotación independiente de las esquinas

Para controlar individualmente cada esquina se definieron ocho variables distintas:

- giroHolocronSup1
- giroHolocronSup2
- giroHolocronSup3
- giroHolocronSup4
- giroHolocronInf1
- giroHolocronInf2
- giroHolocronInf3
- giroHolocronInf4

Los controles utilizados fueron:

Tecla	Pieza
F1	Esquina superior 1
F2	Esquina superior 2
F3	Esquina superior 3
F4	Esquina superior 4
F5	Esquina inferior 1
F6	Esquina inferior 2
F7	Esquina inferior 3
F8	Esquina inferior 4

F9	Mueve todas las esquinas
F10	Regresa todas las esquinas a su posición inicial

Al mantener presionada la tecla Shift, se invierte el sentido del movimiento.

La velocidad utilizada fue: $GLfloat\ velocidadHolocron = 60.0f * deltaTime$; El uso de `deltaTime` permite que la velocidad del movimiento no dependa directamente del número de cuadros por segundo.

Movimiento de las esquinas alrededor del Holocrón

Cada esquina comienza a partir de la matriz `holocronPadre`, por lo que su movimiento se calcula respecto al centro del Holocrón.

Además del giro alrededor del cuerpo, se añadió una pequeña separación progresiva. Esta distancia se obtiene mediante: $\sin(\text{angulo} / 2)^2$ y se multiplica por tres unidades. De esta manera, la esquina comienza cerrada, se separa gradualmente y posteriormente vuelve a su posición original.

Después se agrega su desplazamiento hacia afuera y finalmente una rotación alrededor del centro geométrico de esa pieza, esto mismo se repite de manera independiente para las ocho esquinas

```

623 //triangulosupl: F1; solo esta esquina utiliza giroHolocronSupl
624 aperturaHolocron = std::sin(glm::radians(giroHolocronSupl) * 0.5f);
625 aperturaHolocron = 3.0f * aperturaHolocron * aperturaHolocron;
626 model = glm::rotate(holocronPadre, glm::radians(giroHolocronSupl), glm::vec3(0.0f, 1.0f, 0.0f));
627 model = glm::translate(model, glm::normalize(glm::vec3(-1.0f, 1.0f, -1.0f)) * aperturaHolocron);
628 model = glm::translate(model, origenEsquinas);
629 model = glm::rotate(model, glm::radians(-180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
630 //giro propio alrededor del centro geometrico de esta esquina |
631 glm::vec3 centroSupl = glm::vec3(3.81922685f, 3.73523595f, 3.86784493f);
632 model = glm::translate(model, centroSupl);
633 model = glm::rotate(model, glm::radians(giroHolocronSupl), glm::normalize(glm::vec3(1.0f, 1.0f, 1.0f)));
634 model = glm::translate(model, -centroSupl);
635 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
636 TrianguloSupl_M.RenderModel();

```

Figura 4. Movimiento independiente de una esquina del Holocrón.

Importación jerárquica del satélite

El satélite fue dividido en un cuerpo principal y cuatro alas. La segunda ala de cada lado hereda primero la transformación de la primera.

```

743 //satelite, todo el conjunto hereda su traslacion en X, Y y Z
744 glm::mat4 satelitePadre = glm::translate(glm::mat4(1.0f), posicionSatelite);
745 satelitePadre = glm::scale(satelitePadre, glm::vec3(0.65f));
746 GLfloat escalaSateliteFBX = 0.16809055f;
747 glm::vec3 pivoteAlaDer1 = glm::vec3(1.91174454f, 0.65587784f, 0.58370453f);
748 glm::vec3 pivoteAlaDer2 = glm::vec3(7.38424561f, 2.50917572f, 0.56575607f);
749 glm::vec3 pivoteAlaIzq1 = glm::vec3(-1.91173248f, 0.65587784f, 0.58370453f);
750 glm::vec3 pivoteAlaIzq2 = glm::vec3(-7.38423401f, 2.50917572f, 0.56575607f);
751
752 color = glm::vec3(0.75f, 0.78f, 0.82f);
753 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
754 model = glm::scale(satelitePadre, glm::vec3(escalaSateliteFBX));
755 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
756 CuerpoSatelite_M.RenderModel();
757
758 //jerarquia, cuerpo -> derecha1 -> derecha2; cuerpo -> izquierdal -> izquierda2
759 color = glm::vec3(0.88f, 0.25f, 0.55f);
760 glUniform3fv(uniformColor, 1, glm::value_ptr(color));

```

Figura 5. *Satélite importado utilizando una estructura jerárquica.*

Movimiento del satélite sobre X, Y y Z

Para permitir que todo el satélite se desplazara como un único objeto se utilizó: glm::vec3 posicionSatelite, su posición inicial fue: glm::vec3(18.0f,7.0f,-1.5f);

El movimiento se realiza con los siguientes controles:

Tecla	Movimiento
Flecha derecha	+X
Flecha izquierda	-X
Page Up	+Y
Page Down	-Y
Flecha arriba	-Z
Flecha abajo	Z

La velocidad de movimiento es: GLfloat velocidadSatelite =5.0f * deltaTime;

Todas las piezas del satélite parten de satelitePadre, por lo que al modificar la posición del cuerpo las cuatro alas se desplazan junto con él.

```

758 //jerarquia, cuerpo -> derecha1 -> derecha2; cuerpo -> izquierda1 -> izquierda2
759 color = glm::vec3(0.08f, 0.25f, 0.55f);
760 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
761
762 glm::mat4 alaPadreDer1 = glm::translate(satelitePadre, pivoteAlaDer1);
763 alaPadreDer1 = glm::rotate(alaPadreDer1, glm::radians(giroAlaDer1), glm::vec3(0.0f, 0.0f, 1.0f));
764 model = glm::scale(alaPadreDer1, glm::vec3(escalaSateliteFBX));
765 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
766 AlaDer1_M.RenderModel();
767
768 glm::mat4 alaPadreDer2 = glm::translate(alaPadreDer1, pivoteAlaDer2 - pivoteAlaDer1);
769 alaPadreDer2 = glm::rotate(alaPadreDer2, glm::radians(giroAlaDer2), glm::vec3(0.0f, 0.0f, 1.0f));
770 model = glm::scale(alaPadreDer2, glm::vec3(escalaSateliteFBX));
771 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
772 AlaDer2_M.RenderModel();
773
774 glm::mat4 alaPadreIzq1 = glm::translate(satelitePadre, pivoteAlaIzq1);
775 alaPadreIzq1 = glm::rotate(alaPadreIzq1, glm::radians(-giroAlaIzq1), glm::vec3(0.0f, 0.0f, 1.0f));
776 model = glm::scale(alaPadreIzq1, glm::vec3(escalaSateliteFBX));
777 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
778 AlaIzq1_M.RenderModel();
779
780 glm::mat4 alaPadreIzq2 = glm::translate(alaPadreIzq1, pivoteAlaIzq2 - pivoteAlaIzq1);
781 alaPadreIzq2 = glm::rotate(alaPadreIzq2, glm::radians(-giroAlaIzq2), glm::vec3(0.0f, 0.0f, 1.0f));
782 model = glm::scale(alaPadreIzq2, glm::vec3(escalaSateliteFBX));
783 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
784 AlaIzq2_M.RenderModel();

```

Figura 6. Desplazamiento del satélite sobre los tres ejes.

Rotación de las partes del satélite

Para cumplir con el ejercicio de agregar rotaciones a por lo menos tres piezas, se implementó movimiento en **las cuatro alas**:

- giroAlaDer1
- giroAlaDer2
- giroAlaIzq1
- giroAlaIzq2

Tecla	Movimiento
7	Ala derecha 1
8	Ala derecha 2
9	Ala izquierda 1
0	Ala izquierda 2

La tecla Shift permite invertir el sentido del giro. Cada ala puede girar entre: -90° y $+90^\circ$. El límite se aplica mediante: `glm::clamp()`. Esto evita que los paneles realicen rotaciones poco realistas.

Puntos de articulación de las alas

Una parte importante del ejercicio era que las piezas no giraran alrededor de todo el satélite, sino desde su unión real con el cuerpo.

Para lograrlo se definieron puntos de pivote específicos:

- `pivoteAlaDer1`
- `pivoteAlaDer2`
- `pivoteAlalZq1`
- `pivoteAlalZq2`

Por esto, al mover el primer segmento también se modifica correctamente la posición del segundo. Lo mismo ocurre en el lado izquierdo.

Reinicio del satélite

También se agregó una opción para regresar únicamente el satélite a su estado inicial al presionar: Home, se restablece su posición: `glm::vec3(18.0f, 7.0f, -1.5f)`;

y los cuatro ángulos de las alas vuelven a cero. `practica5`

Esta función facilita regresar rápidamente al estado inicial después de realizar diferentes pruebas de movimiento.

Resultado

Como resultado de la práctica se logró importar correctamente tanto el Holocrón como el satélite utilizando modelos separados.

El Holocrón fue construido mediante un cuerpo central y ocho esquinas independientes. Cada una puede controlarse mediante una tecla diferente, manteniendo como referencia el centro del modelo. También se agregó la posibilidad de mover todas las esquinas simultáneamente y regresarlas a su posición inicial.

En el satélite se consiguió que todo el conjunto se desplazara sobre los ejes X, Y y Z. Además, se agregaron rotaciones independientes en cuatro paneles, superando el mínimo de tres partes solicitado.

La utilización de puntos de pivote permitió que cada panel girara desde su unión correspondiente y no alrededor del centro completo del satélite.

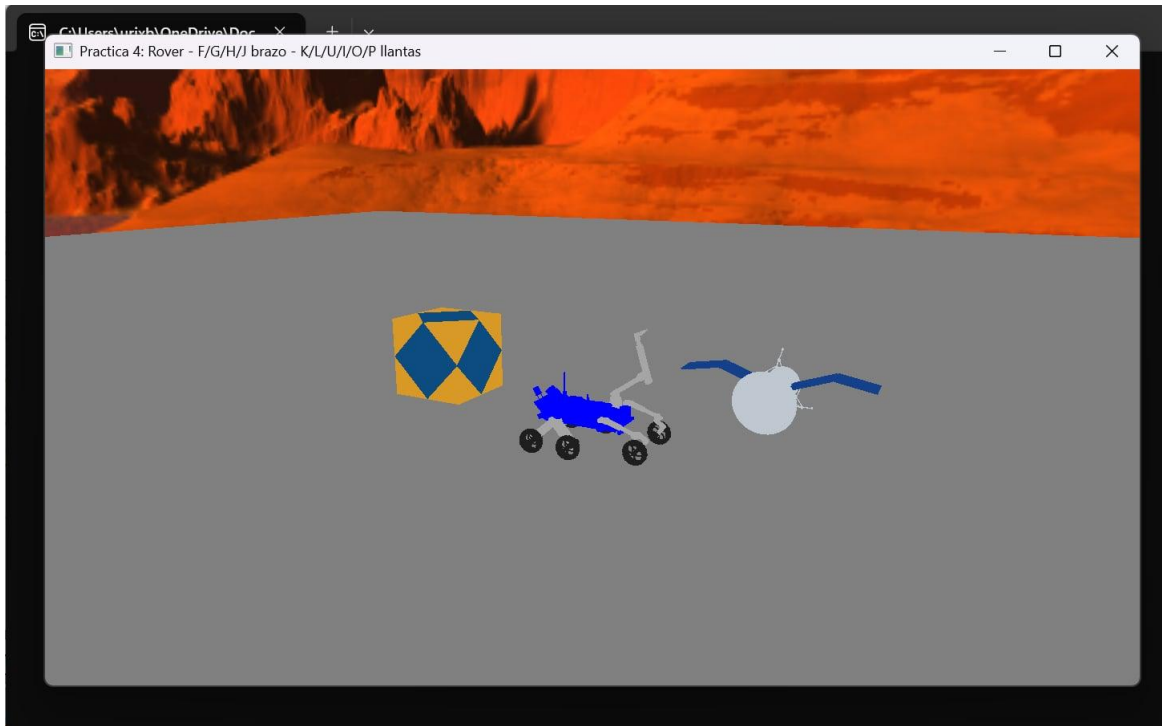


Figura 8. Resultado final de los modelos implementados en la práctica.

Problemas presentados

Uno de los principales problemas que tuve fue preparar correctamente los modelos antes de utilizarlos en el programa. Debido a que las diferentes piezas necesitaban movimientos independientes, fue necesario separarlas previamente en Blender y exportarlas individualmente.

También fue importante colocar correctamente los Origins y puntos de articulación, especialmente en el satélite. Si una de las alas utilizaba un punto incorrecto, podía girar alrededor de todo el modelo en lugar de hacerlo desde su conexión.

Otro aspecto que tuvo que considerarse fue la escala y orientación de los archivos fbx. En el caso del satélite se tuvo que utilizar un factor específico de escala de 0.16809055, para hacerla más grande mientras que en el Holocrón fue necesario corregir su orientación mediante una rotación de -180° sobre Y.

Finalmente, fue necesario organizar correctamente las matrices padre e hijo para evitar que las piezas se separaran cuando el objeto principal cambiaba de posición.

Conclusión:

- a) La parte de mayor complejidad fue el manejo de las transformaciones jerárquicas. En el Holocrón todas las esquinas parten de una matriz central, pero cada una conserva su propio ángulo de movimiento. De esta manera

pueden girar independientemente sin perder su relación con el cuerpo, en el satélite la jerarquía fue más evidente. Por ejemplo, la segunda ala derecha hereda primero el movimiento general del satélite y después la rotación de la primera ala antes de aplicar su propio movimiento

- b) Esta práctica me permitió aplicar el modelado jerárquico en dos situaciones diferentes. El Holocrón utiliza varias piezas distribuidas alrededor de un centro común, mientras que el satélite utiliza una estructura donde algunas piezas son hijas de otras. También se pudo observar la importancia de colocar correctamente los puntos de origen en Blender antes de exportar los modelos, ya que estos puntos determinan posteriormente cómo se comportan las rotaciones. El uso de `deltaTime` permitió mantener movimientos uniformes, mientras que funciones como `glm::clamp()` y `std::fmod()` ayudaron a controlar los rangos de rotación.
- c) Conclusión: Al finalizar esta práctica se concluye que el modelado jerárquico permite organizar y manipular modelos tridimensionales formados por distintas piezas sin perder la relación existente entre ellas. En el Holocrón se logró implementar una estructura formada por un cuerpo principal y ocho esquinas que pueden moverse de manera independiente alrededor del centro. También se agregó un control para moverlas simultáneamente y otro para regresar el modelo a su estado inicial. En el satélite se consiguió realizar desplazamientos sobre los ejes X, Y y Z, manteniendo todas sus partes unidas. Además, se implementaron rotaciones independientes en cuatro paneles, utilizando puntos de pivote ubicados en sus conexiones para conseguir movimientos coherentes con la estructura del modelo.

Bibliografía

- Céslava. (s. f.). *Organización de escena, duplicado y origen del objeto: cómo dejar de trabajar con caos en Blender*. Aprende Desarrollo Web. Recuperado de: <https://cursos.ceslava.com/curso/blender/leccion/blender-organizacion-escena-colecciones-duplicado-y-origen-fundamentos>
- Zavala, F. (2025, 12 de diciembre). *Espacios de coordenadas en OpenGL*. Código en Llamas. Recuperado de: <https://codigoenllamas.com/espacios-de-coordenadas-en-opengl>
- FerhanGames. (2024, 1 de diciembre). *Satellite 01 free low-poly 3D model* [Modelo 3D]. CGTrader. Recuperado de: <https://www.cgtrader.com/free-3d-models/space/space-ship/satellite-01-0a93e7cd-f95a-495c-a42a-94e853590ef6>